SHM-EM: A forecast-aware event management framework for heterogeneous engineering monitoring

Ji'an Liao[a,b], Zifa Wang[a,b*], Dengke Zhao[a,b], Jianming Wang[a,b], Zhaoyan Li[a,b], SiranYang[a]Siran Yang[a,b]

[a] Key Laboratory of Earthquake Engineering and Engineering Vibration, Institute of Engineering Mechanics, China Earthquake Administration, Harbin 150080, China

[b] Key Laboratory Earthquake Disaster Mitigation, Ministry of Emergency Management, Harbin 150080, China

* Corresponding Author: ZifaWang

E-mail addressauthor: Zifa Wang, Zifa@iem.ac.cn.

Full mailing address: Institute of Engineering Mechanics, China Earthquake Administration, No. 29 Xuefu Road, Harbin 150080, China.

Abstract

Engineering-monitoring software typically treats data acquisition, time seriesoften separates observations, forecasting, and event response as separate modules. This separation ties, leaving model inputs to project specific schemas, prevents forecasts from sharing a common engineeringschema-bound and temporal frame, and makes itforecast decisions difficult to incorporate predictions safely into formal event workflows. This paper presentsaudit. SHM-EM, is an open-source framework that treats persisted forecasts as contextualized inputs to engineering-event management. A It contributes three mechanisms: a versioned engineering-semantic data-model contract defines how engineering objects and source fields map to ordered model inputs and prediction targets. Forecasts from multiple models are ; a synchronized and summarized as a software-level project future state. After validation, users may evaluate a rule without creating formal records or execute it through an independent eligibility gate that creates events, responses, and provenance only when all checks pass. A Project Future State for multi-target forecasts; and a controlled forecast-to-event transition that separates audited Evaluate from gated Execute. Validation uses a de-identified excavation case demonstrates rolling multi-target forecasting and end-to-end reproduction., a synthetic bridge workflow fixture, a 15-case matrix comprising one positive control, 12 failure-path cases, and two input-availability controls, runtime measurements, and an event provenance trace. The repository includes the public sample, modelreference workflow integrates six fixed-version point-forecast bundles, database scripts, tests, and reproduction tools. SHM-EM presents124 targets, 40 future steps, and 4,960 prediction rows. Docker/Linux reproduced the logical workflow, but not the exact Windows output hash; no tolerance was applied. The contribution is an auditable software workflow rather than, not a new forecasting algorithm or a claim of predictive generalization.

Keywords: engineering monitoring; structural health monitoring; time-series forecasting; event management; provenance; reproducible research software

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	v1.0.01
C2	Permanent link to code/repository used for this code version	https://github.com/lja666/SHM-EM/releases/tag/v1.0.0 (fixed commit: 1d2ab45e516ef4167c6c4c4265da5b533b2eab78)https://github.com/lja666/SHM-EM/releases/tag/v1.0.1 (fixed commit: d7cba1419145e6c75fe69ad63172af5f5abe5028)
C3	Legal code license	MIT License

Nr	Code metadata description	Metadata
C4	Code versioning system ~~used~~	Git
C5	~~Software code languages~~Languages, tools, and services ~~used~~	Java 8; SQL; TypeScript; Python 3.10; Vue 3; Spring Boot 2.6.13; MyBatis 2.2.2; MySQL 8.4; PyTorch 2.11.0; Vite; ECharts; PowerShell 7; Docker Compose; OpenAPI
C6	Compilation requirements, operating environments, and dependencies	Back end: Java 8~~,~~ and Maven 3.8~~+~~,+; database: MySQL 8.0+; front end: Node.js 20+ and npm; forecasting runtime: Python 3.10 with locked dependencies. ~~The public v1.0.0 reproduction workflow was validated on Windows 10/11 with PowerShell 7.~~Windows 10/11 with PowerShell 7 is the exact-output reference. The exercised Docker Compose Linux path reproduces component checks and the logical six-model-to-provenance workflow, but not a bitwise-identical normalized prediction-output hash.
C7	~~If available, link to developer~~Developer documentation/manual	~~https://github.com/lja666/SHM-EM/tree/v1.0.0/docs~~https://github.com/lja666/SHM-EM/tree/v1.0.1/docs (README, installation, reproduction, models, database, API, security, and data-availability documentation~~).)~~
C8	Support email ~~for questions~~	nlfdzlja@163.com

1. Motivation and significance

~~The main software challenge is fragmentation rather than a lack of sensors or forecasting methods. Structural health~~Engineering monitoring ~~now~~ combines ~~displacement, settlement, strain, pressure, vibration, and environmental~~ heterogeneous measurements [1~~]. Networked acquisition also allows one project to use devices with different sampling and communication characteristics [2,~~3~~], while~~] and increasingly uses machine learning ~~supports anomaly and damage detection~~ [4-6~~]. However, monitoring platforms remain tied to vendor tables, field names, units, and point identifiers, whereas~~], yet applications and model scripts often ~~assume fixed columns~~ retain project-specific tables, units, identifiers, and implicit ~~ordering. These dependencies hinder model integration, interpretation, verification, and reuse across projects.~~

~~Existing standards solve only part of this problem. Sensor Web Enablement and the~~ feature order. Standards such as OGC SensorThings ~~API standardize~~organize sensor ~~and observation concepts and queries~~ observations [7,8], while ~~complex~~ generic CEP provides stream and event~~ processing research supports aggregation, condition matching, and event generation~~ patterns [9]. ~~They do not define model-specific input contracts, a shared future frame for forecasts from multiple models, or the boundary between forecast analysis and formal engineering events. A forecast-aware workflow must therefore preserve engineering semantics when constructing model inputs, align heterogeneous forecasts, and admit them to event workflows only after validation and explicit rule evaluation.~~

~~Related research software~~ GMFAgent addresses ~~complementary concerns. GMFAgent organizes knowledge, data, and tools for~~ ground-motion-field estimation [10], ~~whereas~~and Predictive-SHM ~~supports multi-sensor~~provides ingestion~~ and replaceable~~, model registration, forecasting~~ models~~, visualization, and alerts [11]. SHM-EM ~~focuses on the remaining workflow boundary: the contract between observations and model inputs, the temporal alignment of multiple forecasts, and the controlled creation of formal event records~~instead formalizes the downstream boundary through which persisted forecasts enter auditable engineering-event workflows. It neither replaces these systems nor claims SensorThings conformance. Table 1 compares documented responsibilities; *Not reported* means only that the cited primary source does not explicitly document the capability.

~~Recurrent, convolutional, and Transformer-based~~Table 1. Source-grounded responsibility comparison.

Capability	OGC SensorThings	Generic CEP	Predictive-SHM	SHM-EM
Observation access and semantics	Yes	Partial	Yes	Yes
Model-specific ordered input contract	Not applicable	Not applicable	Partial	Yes
Pluggable forecasting/model adapter	Not applicable	Not applicable	Yes	Yes
Shared prediction origin and future timeline	Not applicable	Not applicable	Not reported	Yes
Project-level future-state aggregation	Not applicable	Not applicable	Not reported	Yes
Rule/event evaluation	Not applicable	Yes	Partial	Yes
Execution-time eligibility recheck	Not applicable	Not reported	Not reported	Yes
Formal event-to-prediction provenance link	Not applicable	Not reported	Not reported	Yes

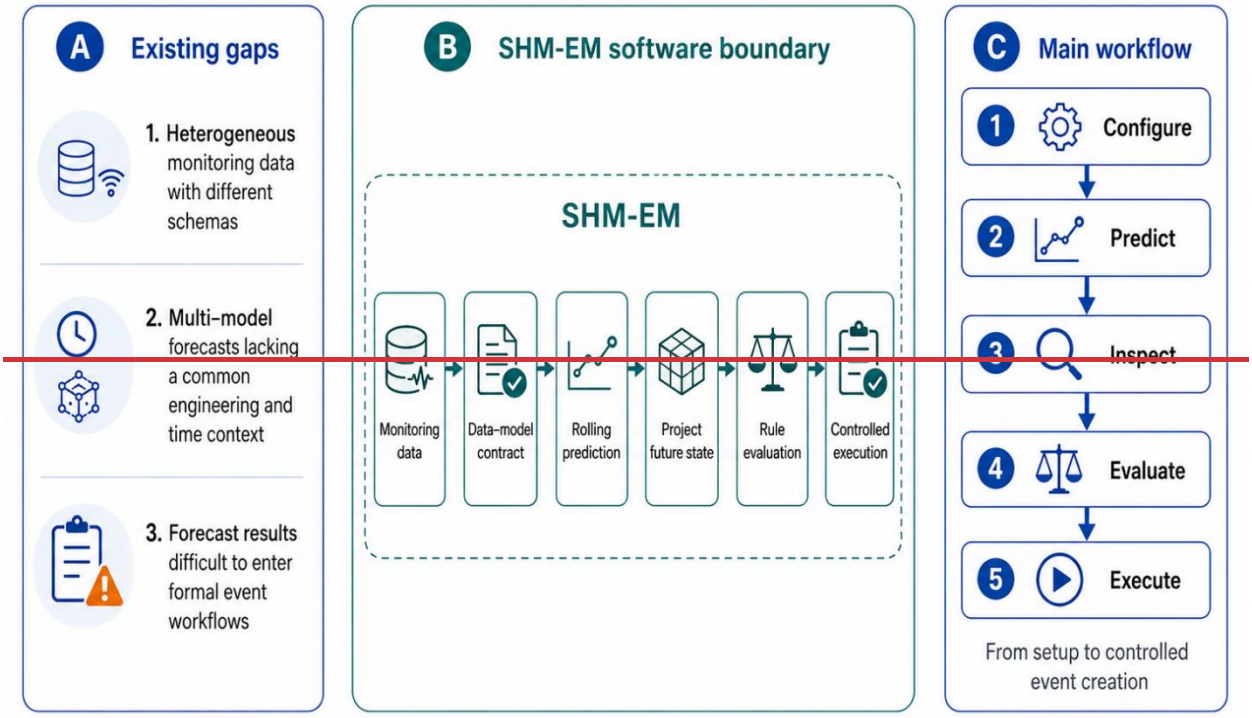
Forecasting methods are ~~widely used for multi-step forecasting~~reviewed elsewhere [12-19]~~. SHM-EM does not compare these algorithms. It examines how~~; SHM-EM registers and governs fixed-version models ~~can be integrated, checked, and used in event management.~~rather than proposing one. Model cards, data documentation, FAIR ~~principles,~~ software practice, citation, and provenance ~~standards guide intended use, publication, reuse, and auditability~~inform its evidence design [20-25]. Its contributions are limited to:

~~SHM-EM combines three mechanisms: a~~ 1. **A versioned engineering-semantic data-model contract**~~, a~~ binding observations, ordered features, targets, units, transformations, model artifacts, temporal settings, and hashes.

2. **A synchronized Project Future State** summarizing target, station, and project ~~future-state representation, and a~~forecast risk on one timeline while separating observed and forecast risk.

3. **A controlled forecast-to-event transition**~~. In this paper~~ in which Evaluate retains an audit run without formal business records, while Execute reloads forecasts and rechecks eligibility, integrity, and rule semantics before creating events and provenance.

Here, *forecast-aware* means that persisted forecasts are aligned, inspected, and conditionally admitted to ~~the event~~a formal workflow rather than used only for visualization~~. Evaluation and execution share rule logic but remain separate API paths and create different records. Fig. 1 is organized in three panels: the research gaps on the left, the SHM-EM software boundary in the centre, and the user workflow on the right~~.



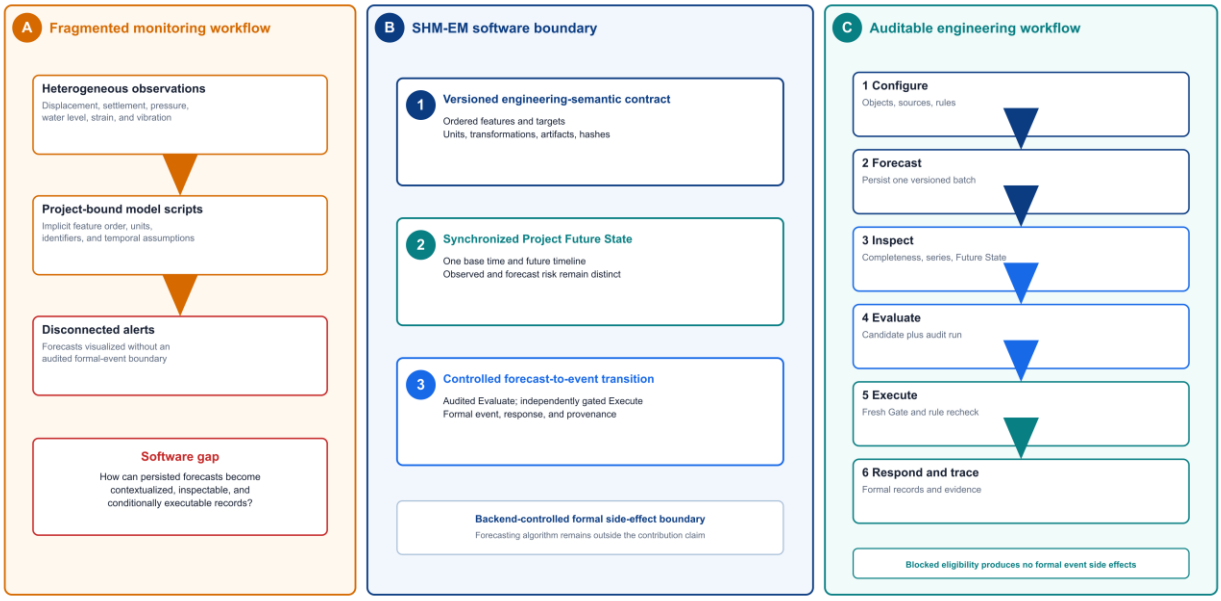


Fig. 1. Research gaps, the SHM-EM software boundary, and the forecast-aware user workflow. OpenAI Codex (model/version unrecorded) assisted layout; authors verified content.

1.1 Intended users and experimental setup

~~SHM-EM is intended for engineering monitoring researchers, data~~Researchers, analysts, and ~~event~~ response personnel~~. Users~~ configure projects, stations, instruments, ~~and monitoring~~ metrics, ~~then bind logical engineering objects to physical data sources through the~~ and approved observation ~~registry. A versioned contract declares model weights, preprocessors, input order, target channels, and temporal settings.~~

~~A typical workflow runs the Python forecasting component~~mappings. PIT_PRE ~~to persist~~persists a prediction batch~~,~~ ~~inspects model runs, the project future state, and observed versus forecast~~; users inspect its completeness, Project Future State, and joint series~~, and then selects evaluation~~ before Evaluate or ~~controlled execution. Section 2.1.4 defines the difference between these paths. The public workflow is reproduced through database~~Execute. Database scripts, APIs, ~~and~~tests, PowerShell~~ rather than through~~, and Docker Compose reproduce the workflow; saved front-end state is not authoritative.

2. Software description

2.1 Software architecture

Fig. 2 ~~organizes SHM-EM into~~shows four layers~~. The upper two layers contain the user interface and application services. The third layer separates the Python forecasting runtime from backend decision logic, and the fourth layer contains persistent storage.~~: Vue ~~supports configuration and inspection~~task views; Spring Boot ~~provides observation, prediction, project future state, and event services; PIT_PRE builds inputs, runs fixed version models, and stores forecasts; MySQL stores~~ services for observations, ~~model registrations, predictions~~conversion, joint series, prediction Gate, Project Future State, rules, events, ~~and~~ responses~~. The right hand callouts emphasize three boundaries: forecasts run outside page interaction, backend services make event decisions, and front-end pages read persisted results.~~

, and provenance; PIT_PRE for contract-driven Python inference; and MySQL persistence. This ~~separation reflects three design choices. It reuses Python~~ separates modelling ~~libraries without adding model~~ dependencies ~~to~~from Java decision services, ~~isolates~~decouples inference from page ~~access~~ ~~frequency~~, and ~~reserves formal~~keeps event creation ~~for~~ backend ~~services~~ controlled. MySQL is the ~~low frequency reference store, while the~~validated implementation; registry and ~~data access~~service interfaces ~~leave~~define an extension ~~points for partitioned, object, or time series storage~~boundary, but no alternative database adapter is validated.

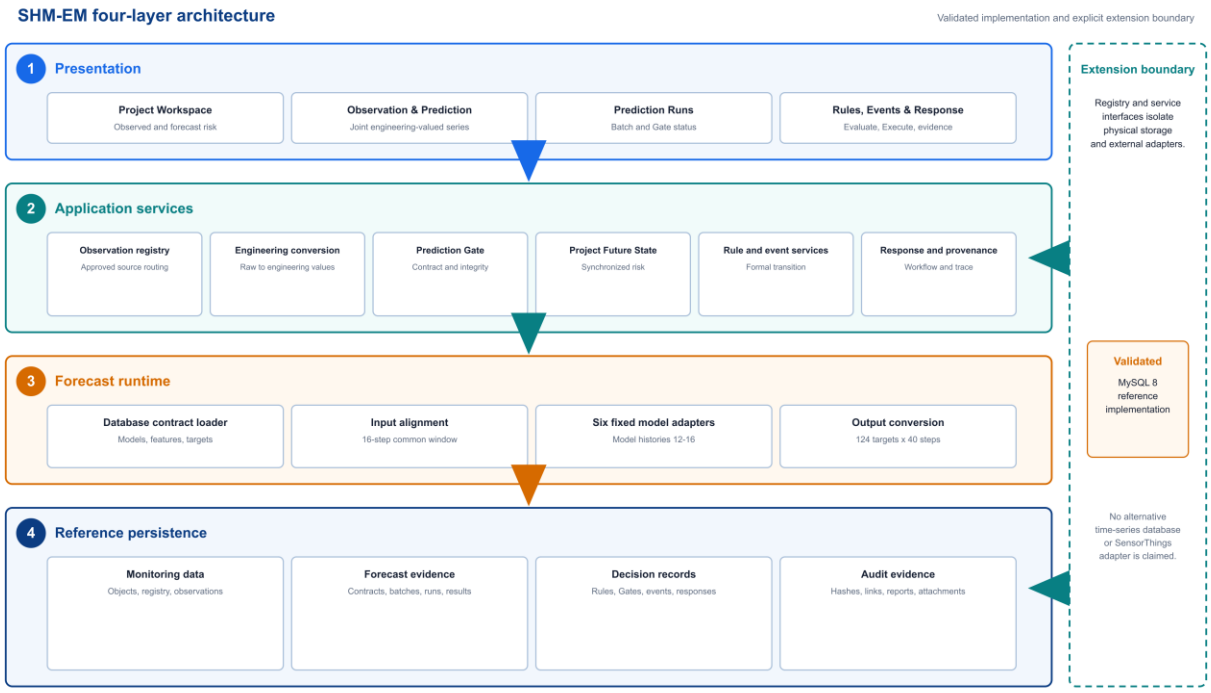
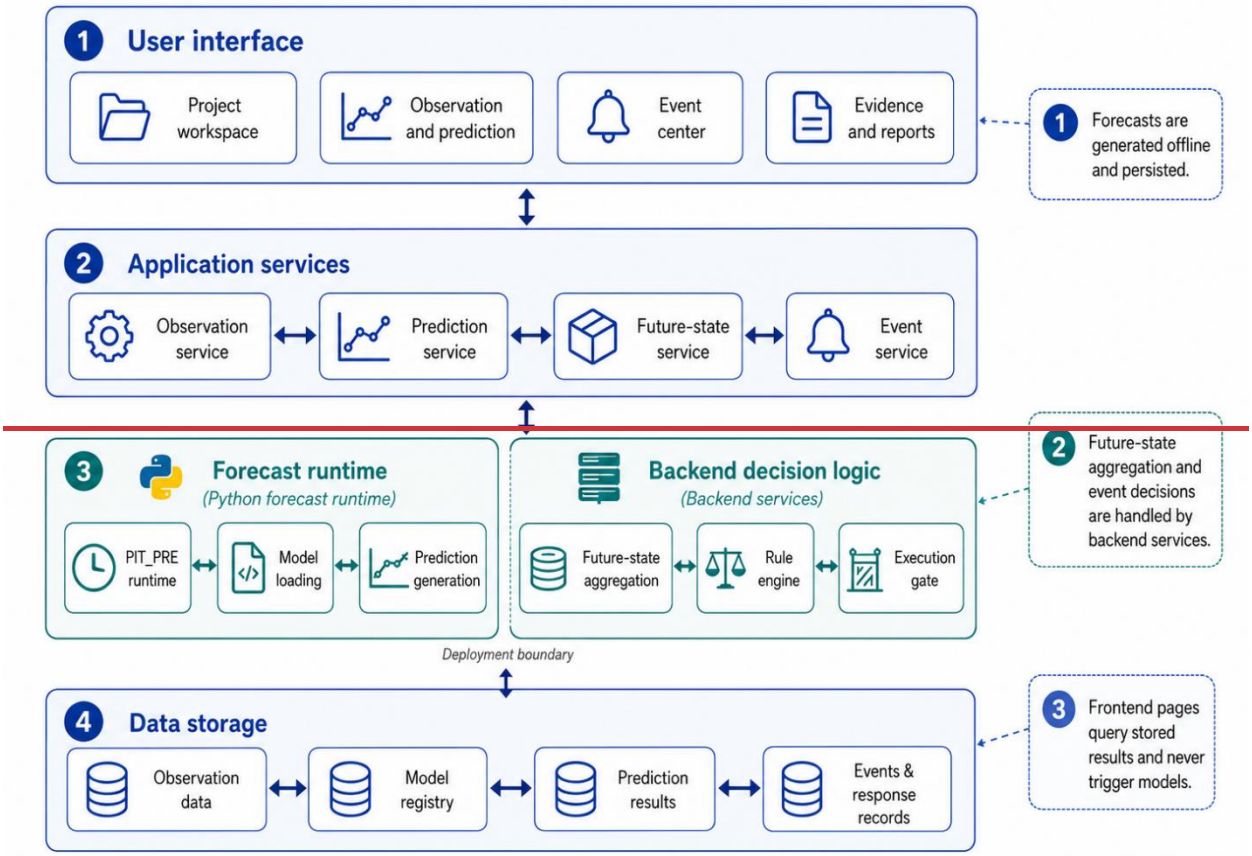


Fig.



~~Fig. 2. Software architecture of the forecast-aware SHM-EM framework.~~

2. Four-layer SHM-EM architecture. MySQL is the validated reference persistence implementation; registry and service interfaces define the storage-adapter boundary. OpenAI Codex (model/version unrecorded) assisted layout; authors checked the software boundaries.

2.1.1 Engineering monitoring object model and observation registry

A stable ~~engineering object~~project-station-instrument-metric model ~~provides the common reference for~~links observations, forecasts, rules, and events~~. The core objects are projects, stations, instruments, monitoring metrics, and observations. Projects define engineering contexts; stations identify locations or components; instruments record device~~:

```
project -> station -> instrument -> metric -> timestamped observation
```

Observations retain source, time, raw and engineering values, unit, quality, and ~~sampling properties; metrics define physical meaning, units, and risk direction. The observation~~conversion provenance. Clients request registry ~~maps logical requests to physical storage, so services do~~and metric codes, not ~~embed~~ table names ~~or project-specific SQL~~. Four typed `em_obs_*` adapters and two registered acceleration partitions are supported; each source must satisfy allowlisted schema, time, unit, and conversion requirements.

2.1.2 Engineering-semantic data-model contract

The ~~data model~~database-authoritative contract ~~defines the linkage among monitoring~~binds objects, ~~source fields,~~ ordered training features, ~~fixed version models, and prediction~~ targets ~~(~~, units, transformations, ~~Fig. 3A). Database registrations store model identity, target, history window, and~~ temporal settings~~. Weights~~, artifacts, preprocessors, ~~inference~~ scripts, ~~parameters, and a~~ runtime ~~manifest form a versioned model bundle whose hashes~~

~~are checked before inference. Feature mappings specify canonical codes, training-time field names, input order, source objects, physical fields, and required status. PIT_PRE rejects bundle, schema, contract, or ordering drift.~~environment, and SHA-256 values. The ~~contract therefore decouples inference from legacy tables and maps model outputs back to explicit engineering objects.~~

~~For example, the canonical feature code point1_settlement_value identifies its project and station context, monitoring metric, physical source field, feature order, transformation, target role, schema version, and model association. The runtime can therefore assemble the model input without embedding project-specific column logic. Model cards and~~ public~~-sample documentation record intended use and limitations [20,21].~~

~~2.1.3 Multi-target rolling forecasts and project future state~~

~~The project future state is a software-level representation of synchronized multi-target forecasts and their risk summaries. Models share one prediction origin and future timeline; outputs are summarized first by target and station and then at project level (~~ contract has 164 ordered source features and 124 targets. Frozen preprocessors select 114 columns for ~~Fig.~~ 3B). ~~A batch identifies one project and origin, each model run records version and input-window metadata, and each result identifies a target channel, future step, horizon, and timestamp. The API representation includes the batch identifier, base time, forecast horizon, target summaries, station summaries, earliest predicted exceedance, policy version, and state hash.~~

~~The reference case integrates six fixed-version models.~~ YD, XD, Strain, Pressure, and Water~~ each use 114 input features;~~, and 164 for Settlement~~ uses all 164. Together they produce 124 target channels and 40 future steps at 3-min intervals. Rules provide thresholds, units, and consecutive-step requirements for target, station, and project summaries. Targets without an applicable rule are marked not evaluated. The project future state supports risk inspection; it neither replaces formal rules nor represents multi-physics joint learning~~. A 16-step common window serves model-specific 12-16-step histories; Pressure declares 13 runner rows and consumes the final 12 (`m=10, lag=2`). Table 2 distinguishes aligned input widths from mapping and output counts.

Table 2. Verified model-bundle and tensor contract.

Model	Engineering target	Required history	Aligned input features	Output targets
YD	Deep horizontal displacement Y (mm)	16	114	42
XD	Deep horizontal displacement X (mm)	12	114	42
Strain	Earth-pressure strain (microstrain)	13	114	14
Pressure	Earth pressure (MPa)	13	114	14
Water	Groundwater elevation (m)	13	114	2
Settlement	Surface settlement (mm)	12	164	10

Full fields, mappings, hashes, and model parameters are repository evidence. Listing 1 shows the manuscript-facing schema subset.

Listing 1. Compact extract from the versioned contract.

```
{
  "contractVersion": "pit_pre_contract_v1",
  "timeline": {"steps": 40, "stepMinutes": 3, "sharedBaseTime": true},
  "model": {"code": "settlement", "history": 12, "inputs": 164, "outputs": 10},
  "feature": {"order": 115, "code": "dtu1_point1_settlement_value",
              "rawUnit": "mm", "engineeringUnit": "mm", "required": true},
```

```
  "inputSchemaSha256": "5c2f6f0f...672daa65f1"
}
```

The schema-validated example and database export are in `docs/revision/examples/` and `artifacts/revision/manuscript/`.

Missing and asynchronous observations

Each required feature is matched backward-asof to a 3-min grid within one cadence; unresolved partial gaps use the declared linear interpolation and boundary-fill policy. Runs record signed source-time offsets, fill counts, ratios, and gap summaries. If an entire required feature is unavailable, or a required value remains unresolved, inference is rejected. Freshness is checked separately: OPERATIONAL uses wall-clock age, whereas REPLAY uses scenario time.

2.1.3 Multi-target rolling forecasts and Project Future State

A batch identifies its project, base time, cadence, horizon, and required models. Runs retain contract, input, artifact, alignment, and integrity metadata; results retain target, step, time, raw and engineering values, unit, conversion, and quality. Project Future State deterministically summarizes synchronized predictions and rule-bound risk without replacing formal rules or implying uncertainty estimation or multi-physics learning.

Algorithm 1. Policy-bound Project Future State aggregation.

```
INPUT project, batch, horizon, mode, reference time
1 Validate the active policy and its canonical hash.
2 Resolve a successful project-owned batch and positive horizon.
3 Inspect Gate eligibility for the requested temporal mode.
4 Load engineering-valued predictions and unit-compatible rule levels.
5 Evaluate each target by step with independent consecutive streaks.
6 Assign the highest activated severity and activation time.
7 Derive forecast risk; merge it with open observed-event risk.
8 Aggregate target, station, and future-timeline summaries.
9 Hash canonical state and return policy, Gate, risks, and summaries.
```

Inclusive operators and `between` include equality; strict operators do not. A nonmatch resets its streak, and severity governs risk ordering. Gate status is reported but does not alter the summary or become a hidden Execute prerequisite.

2.1.4 Controlled transition from forecasts to engineering events

~~After forecasts are stored, SHM-EM validates the batch and prepares a unified engineering series before exposing two independent paths (~~`MetricSeriesPoint` exposes common~~Fig. 3C). Observed and forecast values share~~ object, metric, time, engineering ~~-~~value, unit, quality, source, and provenance fields~~. An execution gate is a backend eligibility check applied immediately before formal records are created. It verifies batch status, required~~; predictions add batch, run, model ~~runs and forecast steps, temporal validity, quality flags, and model bundle hashes. REPLAY uses a scenario clock, OPERATIONAL uses the current clock, and REPRODUCTION supports the isolated public example while suppressing real notifications.~~

, step, conversion, and integrity context. Contract integrity, rule semantics, Evaluate, and Execute ~~use the same rule calculation logic but~~ remain ~~separate API paths.~~distinct. Evaluate inspects a REPLAY Gate, returns ~~candidate events and calculation snapshots and stores an audit run without creating formal events or response records. Execute does not consume a stored~~candidates, and persists one evaluation~~ result. It recalculates the rule, rechecks the execution gate, and creates the~~/audit run, but no formal event, ~~response workflow, and provenance only when all conditions are satisfied.~~Gate record, response, notification, report, evidence, or prediction link. Execute reloads the canonical series, persists a fresh Gate result, revalidates the rule, and only then creates formal records (Fig. 3).

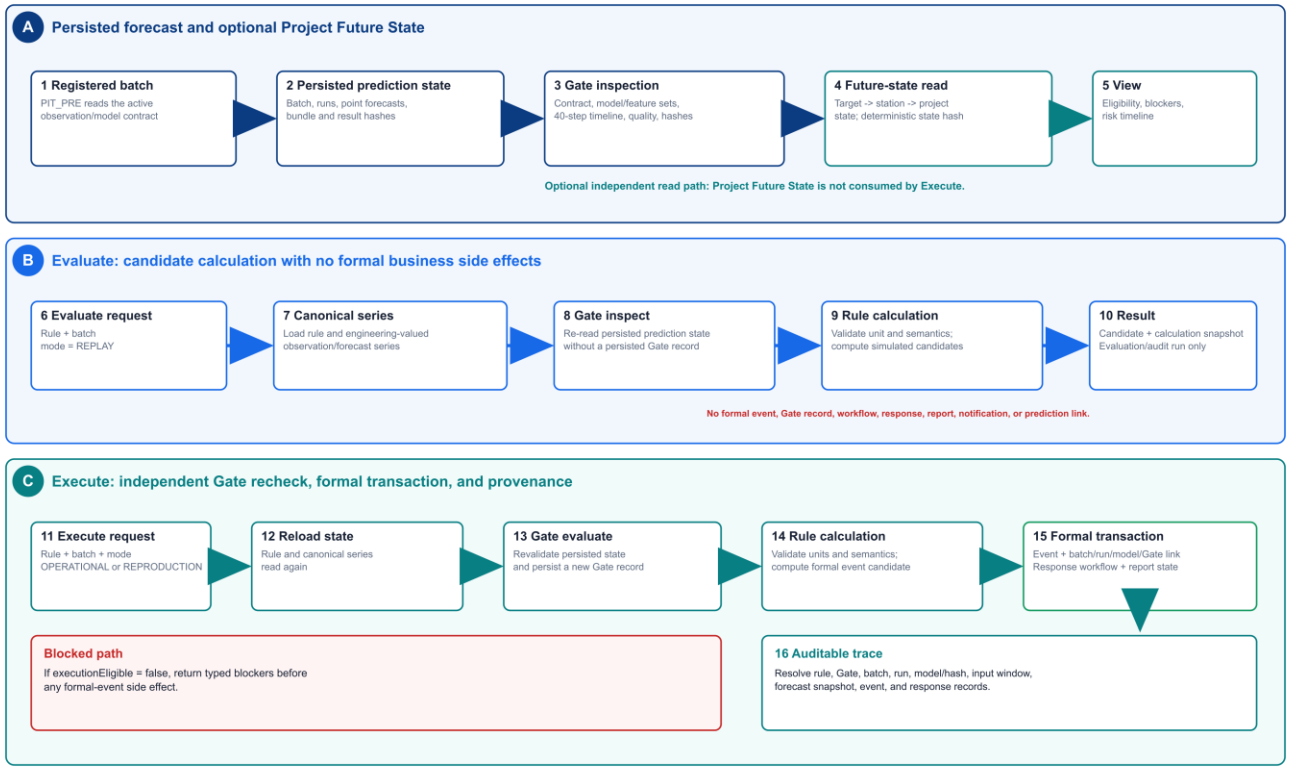
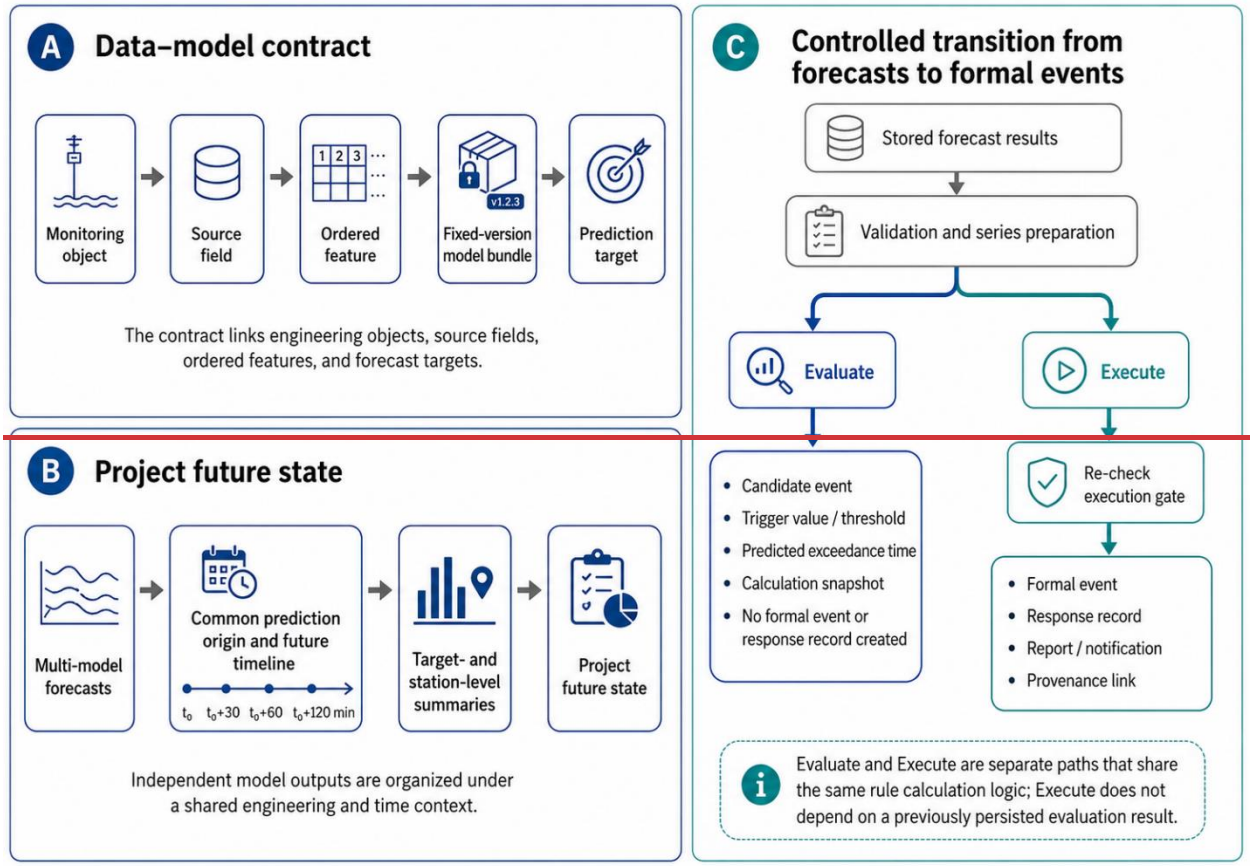


Fig. 3. Controlled sequence from persisted forecasts through optional Project Future State inspection, audited Evaluate, independently gated Execute, and formal provenance. OpenAI Codex (model/version unrecorded) assisted layout; authors checked the code-derived sequence.

2.1.5 Response, provenance, and reproduction

~~A formal event~~Formal events may ~~be linked to a response workflow, notification, report, attachment~~link responses, notifications, reports, generic media attachments, and ~~audit record. For a forecast-driven event, the provenance record also identifies the~~ audits; SHM-EM provides no camera control or capture subsystem. `em_event_prediction_link` records the batch~~, model~~/run, ~~gate result, input window, first~~Gate, exceedance, lead time, ~~predicted~~ peak, consecutive steps, snapshot, and result ~~hash [25]. The current release retains generic media attachments but does not include camera control, video streaming, or automatic screenshots.~~ identity. Reproduction assets ~~are concentrated in~~ include the public ~~repository [22-24,26-30]. They include a de-identified input~~ window, fixed ~~version models and preprocessors, the schema,~~ bundles, database scripts, locked dependencies, tests, ~~PowerShell scripts, and machine-readable results. Inputs, outputs, gates, and project future states are normalized before hashing; environment-specific identifiers are excluded from cross-run hashes.~~entry points, and expected outputs [22-24,26-30].

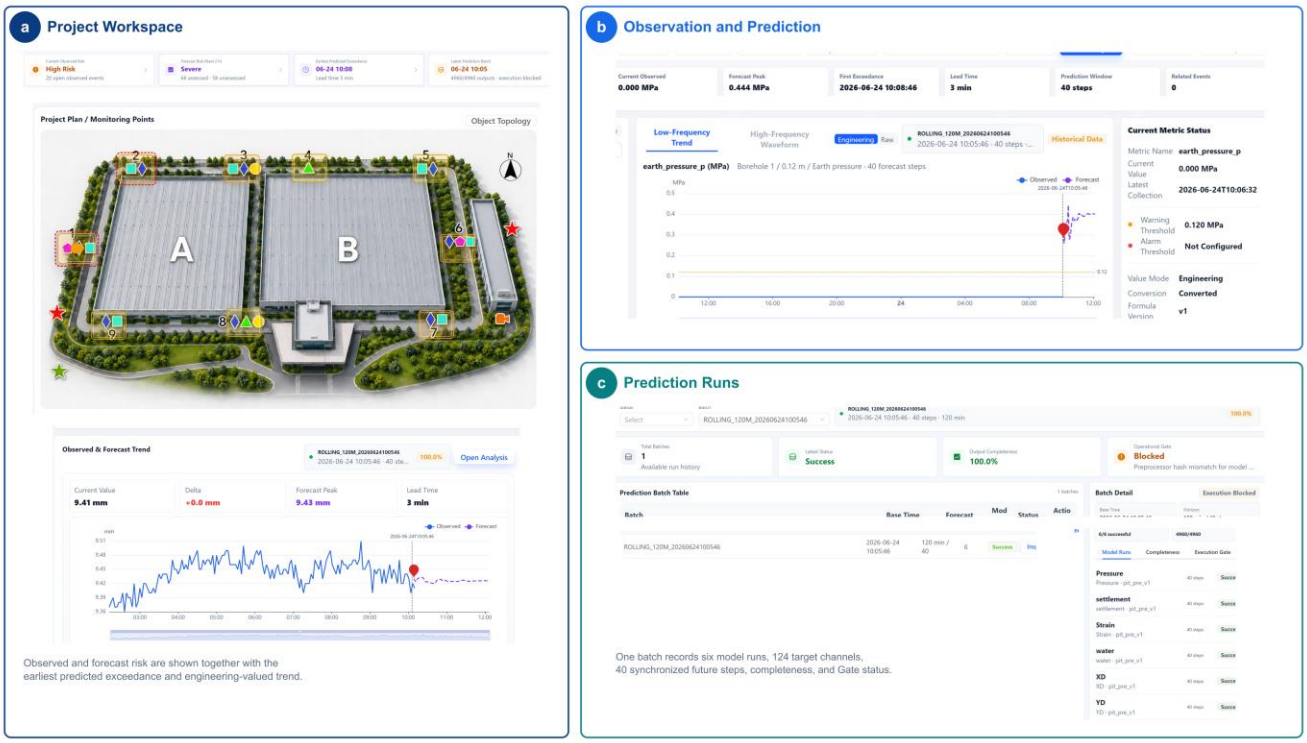


~~Fig. 3. Core mechanisms of SHM-EM.~~

2.2 Main functionalities

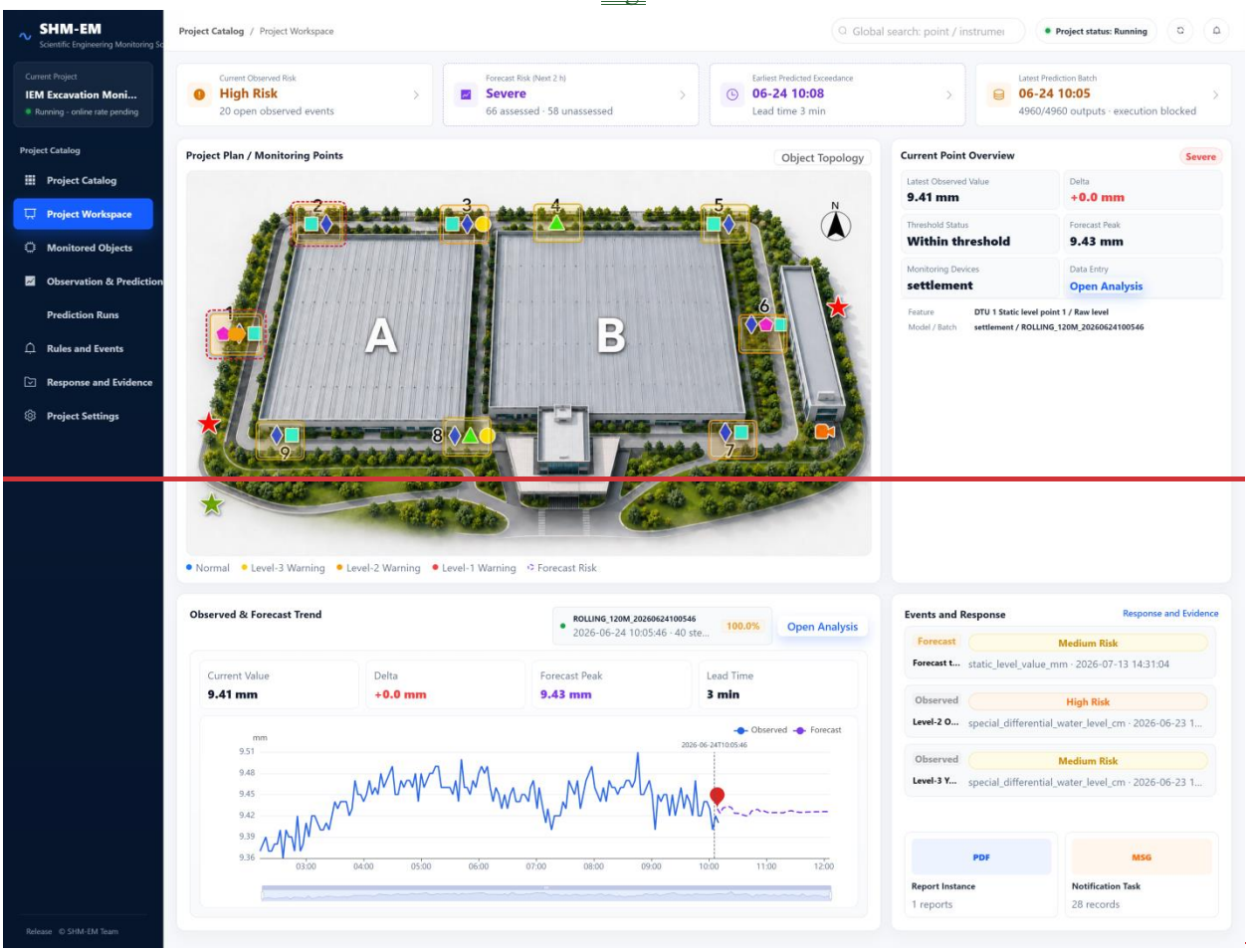
- ~~Project and~~ The interface supports project/observation ~~management: configure engineering objects, map observations to physical storage, and retain engineering values, units, and conversion versions.~~
- ~~Model integration and project future state: register fixed-version models, generate synchronized future series, and summarize risk and predicted exceedance by target, station, and project.~~
- ~~Rule evaluation, execution, and provenance: inspect candidate events without creating formal records, execute eligible rules, organize responses, and trace models, batches, gates, rules, and events.~~
- ~~Public reproduction: rebuild the reference workflow from a de-identified sample, locked dependencies, tests, PowerShell scripts, and SHA-256 checks.~~

~~Fig. 4 presents the three interfaces that best represent the workflow at different scales. Project Workspace (Fig. 4a) provides the project level overview, including the site, current status, project future state, and trend entry point.~~registration, rolling predictions and Future State, Observation ~~and~~ /Prediction ~~(Fig. 4b) supports signal-level inspection by placing observations, forecasts, thresholds, and exceedances on one timeline. Prediction Runs (Fig. 4c) provides run-level~~ rules with controlled execution, response/evidence management, and reproduction/audit. Fig. ~~through~~ 4 combines the Project Workspace, joint series, and prediction-batch ~~status, model completion, output completeness, gate status, and diagnostics. The interfaces use a de-identified configuration; identifiers such as point1_settlement_value~~views; these panels are ~~canonical contract codes rather than device names~~illustrative, not validation evidence.

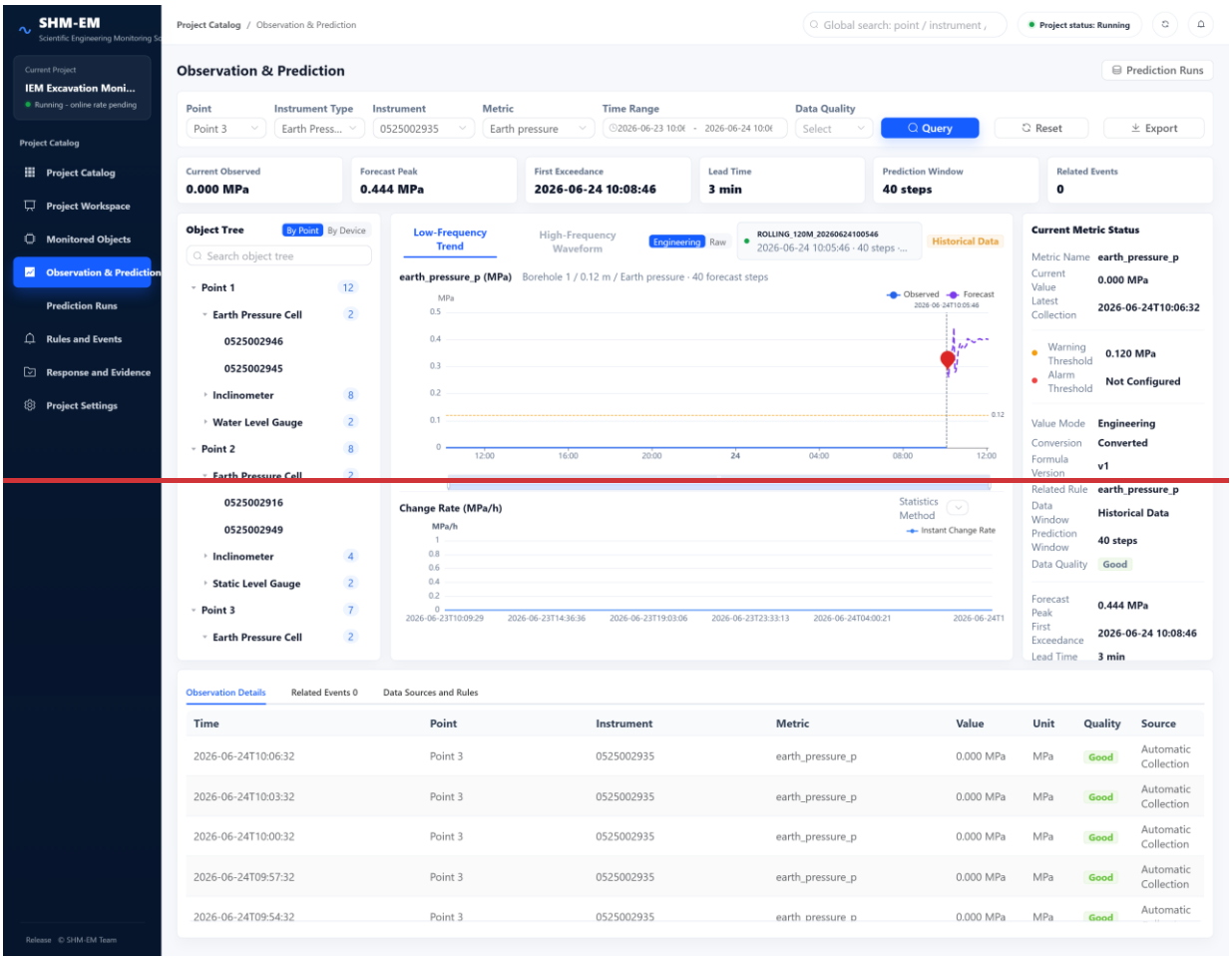

a
Project Workspace
High Risk
Severe
06-24 10:08
06-24 10:05
Project Plan / Monitoring Points
Object Topology
A
B
Observed & Forecast Trend
Open Analysis
Current Value
9.41 mm
Delta
+0.0 mm
Forecast Peak
9.43 mm
Lead Time
3 min
Observed and forecast risk are shown together with the
earliest predicted exceedance and engineering-valued trend.
b
Observation and Prediction
Current Observed
0.000 MPa
Forecast Peak
0.444 MPa
First Exceedance
2026-06-24 10:08:46
Lead Time
3 min
Prediction Window
40 steps
Related Events
0
Low-Frequency Trend
High-Frequency Waveform
Engineering
Raw
Historical Data
earth_pressure_p (MPa)
Borehole 1 / 0.12 m / Earth pressure · 40 forecast steps
Observed
Forecast
Current Metric Status
Metric Name
earth_pressure_p
Current Value
0.000 MPa
Latest Collection
2026-06-24T10:06:32
Warning Threshold
0.120 MPa
Alarm Threshold
Not Configured
Value Mode
Engineering
Conversion Formula Version
Converted
v1
c
Prediction Runs
ROLLING_120M_20260624100546
Total Batches
1
Available run history
Latest Status
Success
Output Completeness
100.0%
Operational Gate
Blocked
Preprocessor hash mismatch for model ...
Prediction Batch Table
Batch
Base Time
Forecast
Mod
Status
Batch Detail
Execution Blocked
6/6 successful
4900/4900
Model Runs
Completeness
Execution Gate
Pressure
settlement
Strain
water
XD
YD
One batch records six model runs, 124 target channels,
40 synchronized future steps, completeness, and Gate status.

235

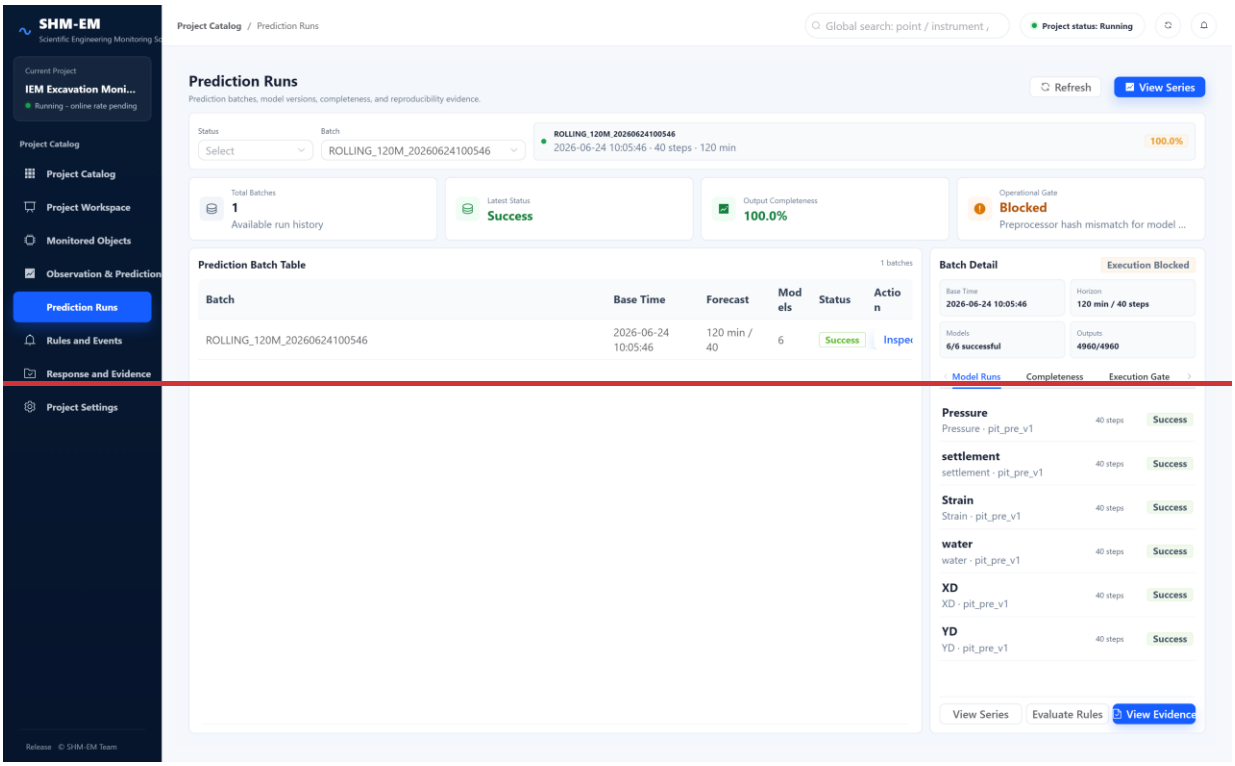
Fig.



(a) Project Workspace



~~(b) Observation and Prediction~~



~~(c) Prediction Runs~~

~~Fig. 4. Task-oriented frontend of SHM-EM~~

4. SHM-EM views of (a) project risk, (b) a joint observation/forecast series, and (c) batch completeness and eligibility. OpenAI Codex (model/version unrecorded) assisted cropping/composition; OpenAI image editing (model/version unrecorded) assisted label removal/upscaling in (a). The panels derive from application captures, and authors verified displayed content.

2.3 Rule configuration and interfaces

~~SHM-EM exposes APIs for project future state queries, rule evaluation, controlled execution, and provenance. Rules are versioned database records and can be configured through the Rules and Events interface~~Versioned rules define source (`OBSERVATION` or ~~through the API. Section 2.1.4 defines the independent Evaluate and Execute paths; full endpoint schemas are provided in OpenAPI.~~

~~A rule specifies a~~`PREDICTION`), metric, operator, threshold~~, minimum~~/unit, consecutive steps, ~~and~~ severity~~.~~, and temporal scope. The ~~simplified example below generates a WARNING candidate when settlement exceeds 54.0 mm for two~~ engine supports latest, extrema, mean, rate, absolute, interval, baseline-relative, and consecutive ~~future steps~~conditions for one metric. Cross-metric compound rules are outside v1.0.1. OpenAPI documents Future State, Evaluate, Execute, and provenance endpoints.

3. Software validation

Independent evidence families are reported separately because their behavior overlaps.

Table 3. Software testing evidence.

Test family	Cases/checks	Passed	Status
Backend unit/service/API tests	55	55	PASS

Test family	Cases/checks	Passed	Status
PIT_PRE contract/alignment/integrity tests	13	13	PASS
Validation matrix (P00, F01-F12, I01-I02)	15	15	PASS
Second-configuration end-to-end acceptance	7	7	PASS
Front-end typecheck and production build	{ "metricCode": "settlement", "operator": "GREATER_THAN", "threshold": 54.0, "minimumConsecutiveSteps": 2, "severity": "WARNING" }2	2	PASS
Public reference end-to-end reproduction	1	1	PASS

The rule engine supports latest, maximum, minimum, mean, rate-of-change, absolute, interval, baseline-relative, and consecutive-step conditions for one metric. The public case uses threshold and consecutive-step rules; cross-metric compound rules are outside the scope of v1.0.0.

3. Illustrative examples

No coverage percentage is claimed because no stable coverage instrument is included.

3.1 Public excavation-monitoring reference case

Fig. 5 combines the reference layout, forecast configuration, and reproduction checks. `SHM_EM_PUBLIC_SAMPLE` is derived from an operational excavation project and retains the minimum provides 2,464 de-identified, time-shifted window required for reproduction. It contains 2,464 low-frequency observations from nine field points, covering for displacement, earth pressure, groundwater level, and hydrostatic level. Coordinates and original device , operational identifiers were , location, and events are removed, and no formal records are preloaded.. Sixteen synchronized3-min steps formsupport the minimum12-16-step model historyhistories. The sample verifies the software chain rather than model accuracy. Fig. 5a shows the monitoring categories; vibration is not included in the public forecast workflow, and the layout is schematic and not to scale.

All six reference models use a Transformer-CNN architecture. Engineering data-use agreements and project confidentiality prevent release of the complete training data, raw time series, and point-level splits. The public model cards report data categories and ranges, split principles, preprocessing, aggregate validation metrics, input and output dimensions, hashes, and known limitations. These summaries define the operating envelope without exposing restricted records. The public sample is only anreproduces inference and reproduction window and is not used to claimsoftware behavior, not independent accuracy or cross-project transferabilitygeneralization. Its reference batch completed six models, 124 targets, 40 steps, and 4,960 rows through conversion, Gate, Future State, Evaluate, Execute, response, and provenance.

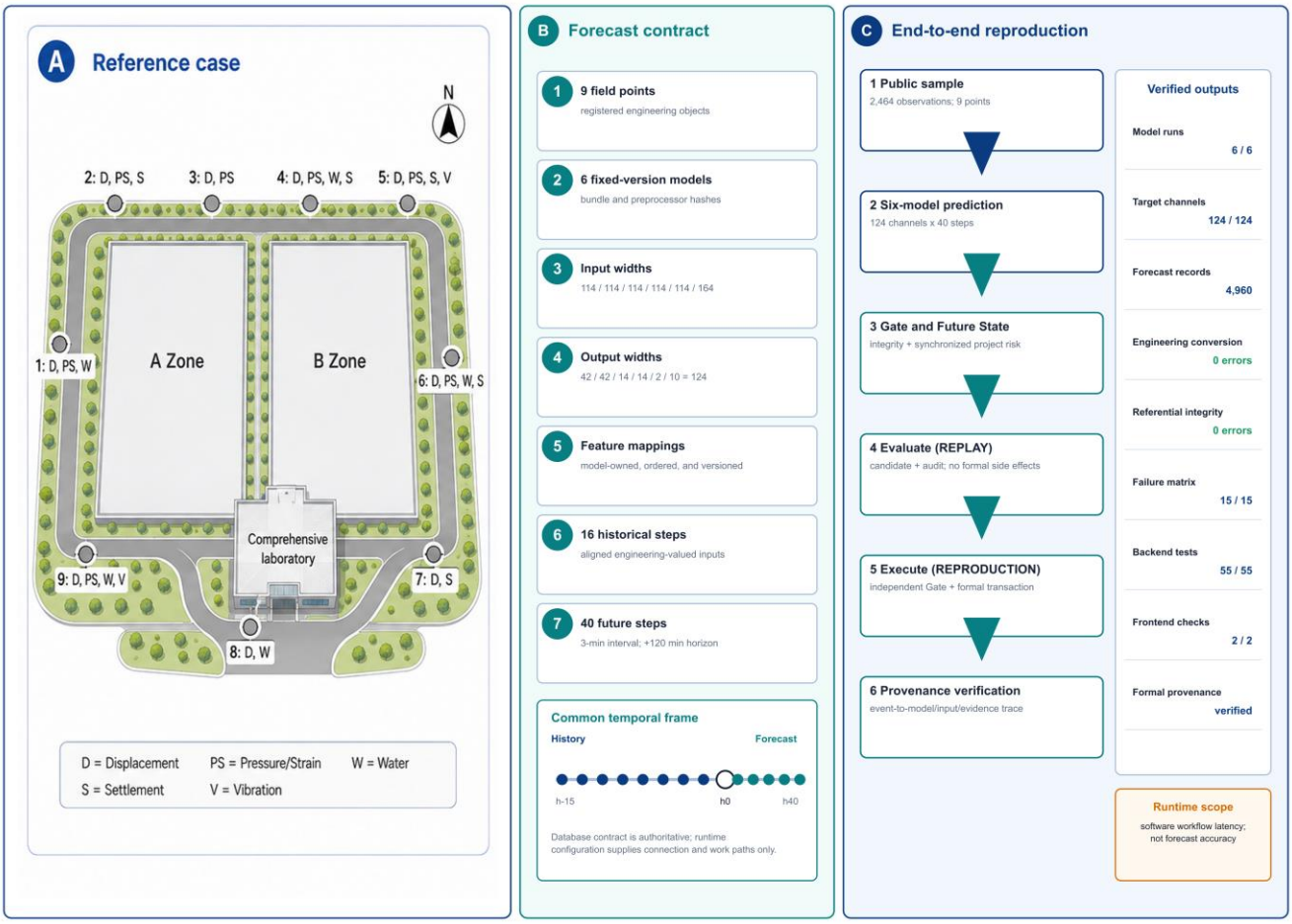


Fig. ~~Table 1. Input and output contracts of the six reference models.~~

5. Public reference case, six-model contract, common timeline, and reproduction checks. The left conceptual panel used OpenAI ChatGPT image generation (model/version unrecorded); OpenAI Codex assisted composition. Authors verified technical labels.

3.2 Cross-configuration reuse

One synthetic bridge configuration, used solely as a software-workflow fixture, tested registration and the end-to-end software boundary; it is not a bridge forecasting study.

Table 4. Synthetic bridge software-reuse fixture.

~~Model~~**Registered or produced item**	~~Engineering output~~**Count/result**	~~Input features~~	~~Output targets~~	~~Primary observation source~~
~~YD~~Stations	~~Deep horizontal displacement Y (mm)~~3	~~114~~	~~42~~	~~Biaxial inclinometer~~
~~XD~~Instruments	~~Deep horizontal displacement X (mm)~~12	~~114~~	~~42~~	~~Biaxial inclinometer~~
~~Strain~~Compatible model bundles	~~Earth-pressure strain (με)~~2	~~114~~	~~14~~	~~Earth-pressure cell~~
~~Pressure~~Persisted forecast rows	~~Earth pressure (MPa)~~1,120	~~114~~	~~14~~	~~Earth-pressure cell~~
~~Water~~End-to-end functional checks	~~Groundwater elevation (m)~~7/7 PASS	~~114~~	~~2~~	~~Differential-pressure water-level sensor~~

~~Model~~Registered or produced item	~~Engineering output~~Count/result	~~Input features~~	~~Output targets~~	~~Primary observation source~~
~~Settlement~~Frozen core/schema modifications	~~Surface settlement (mm)~~0	~~164~~	~~10~~	~~Hydrostatic levelling sensor~~

~~3.2 Synchronized rolling forecasts and future-state construction~~

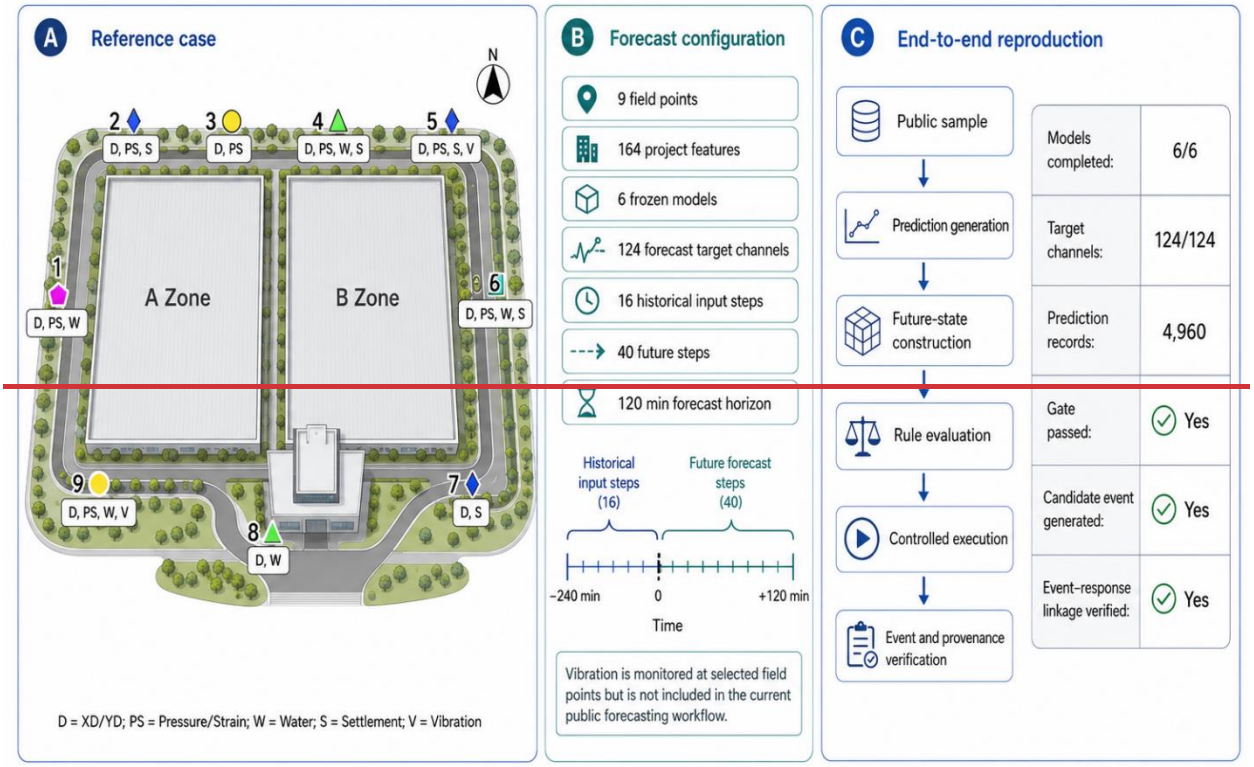
~~PIT_PRE aligns observations from physical tables to common sampling times and selects 114 or 164 ordered features according to each contract. Fixed weights and preprocessors produce native outputs that are converted to versioned engineering quantities. The reference batch contains six successful runs, 124 target channels, 40 future steps, and 4,960 forecast records. The gate checks model and target sets, the timeline, quality flags, and bundle hashes; the project future state service then produces target summaries, station contributions, and a 120 min risk timeline. Fig. 5b summarizes the scale and temporal settings.~~

~~3.3 Rule evaluation and controlled execution~~

~~The reproduction test exercises both paths in sequence: it calls Evaluate in REPLAY mode and then Execute in REPRODUCTION mode within an isolated database. This order verifies their separation; Execute remains independent of the stored evaluation result. REPLAY returns candidates without changing formal records. REPRODUCTION recalculates the rule, rechecks the gate, and creates the event, response, report, and provenance while suppressing real notifications.~~

~~3.4 Scripted reproduction~~

~~The script reproduce-local.ps1 initializes the database, runs component tests and six model inference, starts the API, and verifies the workflow. Back-end tests, front-end checks and build, and PIT_PRE contract tests passed. The models generated 4,960 synchronized records with no conversion or referential integrity errors. Project future state construction, evaluation, controlled execution, provenance, and response matched the expected results. SHA-256 verifies fixed inputs and outputs, while workflows with runtime identifiers use normalized within run checks (Fig. 5c).~~



~~Fig. 5. Public reference case, forecast configuration, and end-to-end reproduction.~~

The fixture produced 1,120 rows; integrity, Gate, Future State, Evaluate, Execute, response, provenance, API, and interface checks passed, while an unavailable mapping was rejected. Outputs are not interpreted as bridge-domain predictive validation or cross-domain accuracy. The result supports this tested configuration only.

3.3 Failure-path and execution-safety validation

A 15-case validation matrix comprising one positive control, 12 failure-path cases, and two input-availability controls ran in isolated databases. Blocked cases produced no formal event, response, report, evidence, or prediction link.

Table 5. Validation matrix and side-effect boundary.

Group	Cases	Condition	Expected/observed boundary
Positive control	P00	Valid reference prediction and rule	Execute succeeds; one formal event is created
Contract/model/batch	F01, F03, F04, F06, F08, F11	Incomplete steps; artifact hash mismatch; missing target; schema mismatch; failed run; failed batch	Execution Gate blocks
Timeline/quality/integrity	F02, F07, F09, F10	Stale prediction; temporal misalignment; corrupted persisted values with stale hashes; invalid quality	Gate or persisted-result integrity blocks
Rule semantics	F05	Incompatible engineering unit	Rule validation blocks
Evaluate-to-Execute mutation	F12	Persisted state is changed after Evaluate	Execute reload/recheck blocks

Group	Cases	Condition	Expected/observed boundary
Input availability	I01, I02	Partial dropout; entire required feature unavailable	Declared alignment policy resolves the partial gap; unavailable required feature rejects input assembly

F09 motivated canonical-row integrity recomputation; F12 confirms that Execute reloads and rechecks state after Evaluate. This validates the tested fail-closed boundaries, not absolute safety.

3.4 Runtime and bounded scalability

Table 6 reports median/p95 Windows reference measurements from the fixed implementation. These are single-process reproduction workloads, not production-throughput tests.

Table 6. Selected runtime and bounded-scaling evidence (ms).

Operation	Workload	n	Median	p95
Full prediction batch	6 models; 4,960 rows	30	16,778.359	18,729.326
Execution Gate	4,960 rows	30	343.129	407.100
Project Future State	Reference batch	30	472.342	574.761
Rule Evaluate	Reference batch	30	269.465	313.340
Rule Execute	Reference batch	10	317.238	336.361
Provenance trace	One event	30	2.578	20.692
Gate S1	4,960 rows	10	2,406.939	2,666.804
Gate S2	49,600 rows	10	3,603.382	3,843.174

S1 and S2 are bounded endpoints, not evidence of linear scaling. The 50,000-row Gate cap is an application safeguard, not a MySQL capacity limit; no alternative time-series backend was measured.

3.5 Provenance and reproducibility

Table 7 summarizes one formal event trace; the complete 40-step chain is machine-readable in the repository.

Table 7. Concrete forecast-event provenance trace.

Trace element	Captured value
Formal event/rule	FEVT-4-f61b7667dcc01721aa2a; PRED_GROUND_SETTLEMENT_WARNING v2
Batch/run/model	Batch 40; run 236; settlement pit_pre_v1
Input/model integrity	Input-schema and model-artifact SHA-256 retained
Exceedance/Gate	3-min lead; 9.43204345-mm peak; Gate 1 eligible
Formal records	Event, workflow, four steps, report, prediction link

Windows 10/11 with PowerShell 7 is the exact-output reference. Docker/Linux completed the logical six-model -> 4,960 results -> Gate -> Future State -> Evaluate -> Execute -> provenance path, with structural target/step and contract hashes matching. Its normalized output hash differed (`exactPredictionReproduction=false`);

maximum absolute difference was 0.00285349, `toleranceApplied=false`, and the full row-wise comparison is retained. Native Ubuntu-host execution was not captured.

Revision-stage code edits assisted by OpenAI Codex (model/version unrecorded) were human-reviewed and subjected to the regression and reproduction checks reported here.

4. Impact

4.1 ~~**Research use without workflow changes**~~

~~Researchers can compare model versions, thresholds,~~Reproducible and ~~rule settings without rewriting the acquisition,~~auditable forecast~~, or event handling workflow. The same persisted observation window can be replayed with a selected model bundle or rule version, while Evaluate leaves formal event records unchanged. Normalized outputs and hashes identify whether a model, rule, or software version changes the result.~~ integration

~~**4.2 Reuse through minimal registration**~~

~~Reuse does not require changes to the core back end, front end, event workflow, or existing source tables. For a new project, users only register the engineering objects and define a small set of mappings from source fields to standardized metrics, together with sampling metadata. A new model is added by registering its fixed bundle, ordered inputs, targets, and temporal settings. The existing query, alignment, execution gate, project future state, rule, provenance, and response services remain unchanged. In the reference case, six models with 114- or 164-feature inputs passed through this common workflow and produced 4,960 synchronized forecast records. Public data, locked dependencies, model cards, scripts, and hashes support independent inspection [22–24,26–30].~~

~~**4.3 Routine inspection without code-level intervention**~~

~~After this one-time registration, routine users work through the web interface rather than model scripts or database tables. From a project or monitoring object, they can inspect observed and forecast series, review threshold exceedance and lead time, and distinguish a candidate event from a formally executed event. The interface also shows eligibility or blocking reasons, model versions, input windows, and hashes needed to trace a result.~~

The contract, deterministic public window, expected outputs, tests, failure matrix, runtime records, and provenance make a model run inspectable rather than an implicit script invocation. Evidence supports contract checking, workflow reproduction, and traceability for the tested configurations, not forecasting superiority, calibrated uncertainty, production throughput, or reliability improvement.

4.2 Cross-configuration reuse

The bridge fixture registered three stations, 12 instruments, and two compatible bundles, produced 1,120 rows, passed 7/7 checks, and required zero frozen-core or schema modifications. This supports reuse for one fixture, not bridge accuracy, model transfer, arbitrary tables, or universal no-code onboarding.

4.3 Controlled event transition and traceability

Evaluate and Execute share engineering-valued rule calculations but differ at the formal side-effect boundary. The failure matrix shows blocked execution without formal records; the successful trace resolves event, rule, batch, model, input, Gate, forecast, and response. SHA-256 detects accidental or uncoordinated mutation, not a privileged attacker changing both data and hashes.

4.4 Current deployment and scientific scope ~~**and limitations**~~

~~The current release has been examined in one excavation case; its interface boundaries do not constitute validation for other structures. The reference models provide~~The six bundles emit point forecasts~~, and the public sample is a minimum inference window. Human review, explicit model metadata, and provenance reduce the~~; Gate status is

data/artifact/timeline/quality/freshness eligibility, not uncertainty or probabilistic risk ~~of treating forecasts as safety conclusions. Project future states summarize single metric thresholds and consecutive steps; compound conditions remain with the formal rule engine.~~. MySQL ~~suits the low-frequency case, whereas high-frequency waveforms may require other storage adapters. Reproduction is~~ 8 is the only validated ~~only on Windows 10/11 with PowerShell 7. The public baseline assumes a trusted network and no~~ backend, the 50,000-row cap is application-level, and no SensorThings adapter or conformance result exists. The bridge fixture is not field validation, and arbitrary tables or models may require adapter work. Docker/Linux demonstrated logical, not bitwise, portability.

SHM-EM is a research reference without application authentication~~; Internet deployment~~. Production use requires TLS, external ~~security~~ identity, role-based and separate Execute authorization, least-privilege database access, secret management, protected audit storage, network controls~~.~~, rate limits, and tested backup/restore. Forecasts require engineering review and must not be the sole basis for automated safety decisions.

5. Conclusions

~~SHM-EM integrates heterogeneous observations, fixed-version models, synchronized project future states, rule decisions, and response records. Its data-model contract makes input mappings inspectable, while independent evaluation and execution paths separate candidate analysis from formal event creation. The public excavation case reproduced multi-target forecasting, project-future-state construction, controlled execution, and provenance from nine field points. Current evidence is limited to one domain and point forecasts; future work will extend storage adapters, compound rules, and validation in additional engineering settings.~~

SHM-EM contributes a versioned engineering-semantic data-model contract, a synchronized Project Future State, and a controlled forecast-to-event transition. The public case, workflow fixture, failure matrix, runtime evidence, provenance trace, and bounded container result support auditable integration for tested configurations. They do not establish predictive generalization, calibrated uncertainty, production security, backend neutrality, or exact cross-platform numerical reproduction.

6. Data and software availability

SHM-EM v1.0~~.0 is available from the public GitHub repository (https://github.com/lja666/SHM-EM) and the versioned~~ 1 is archived at https://github.com/lja666/SHM-EM/releases/tag/v1.0.1, fixed release ~~(https://github.com/lja666/~~commit `d7cba1419145e6c75fe69ad63172af5f5abe5028`. `SHM-EM~~/releases/tag/~~-v1~~.0~~.0~~).~~ ~~The source code~~.1.zip has SHA-256 `ea0973b7c82e06c3c8910ec36fcf2c3d47765a87d11552337a86c69de41a7cef`. Code and six ~~fixed-version~~ model bundles ~~are licensed under~~use MIT; the de-identified ~~public~~ sample and conceptual ~~site~~ plan ~~are licensed under~~use CC BY 4.0. The ~~public~~ sample supports ~~reproduction of engineering conversion, six~~ the reported workflow but not model ~~inference, forecast gating, rule evaluation, formal events, responses, and provenance, but is not an independent generalization test. The complete~~generalization. Restricted historical field data ~~include project location, original device~~, locations, operational identifiers, ~~operational records, and continuous monitoring data subject to ownership and contractual restrictions; they cannot be released. The SHA-256 checksum of the v1.0.0 release package is d611601691790346ecb939a7b62fae02be5debb2b5538dbb38de26f8f1ee2a46~~credentials, generated databases, and local state are excluded; the repository documents this boundary and provides tests and expected outputs.

Acknowledgements

The authors thank the members of the research group for their contributions to software development and manuscript preparation. This work was supported by the Scientific Research Fund of the Institute of Engineering Mechanics, China Earthquake Administration (No. 2025B07), the National Natural Science Foundation of China

(Nos. 52378543 and 52378544), and the National Key Research and Development Program of China (No. 2023YFC3805203).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation and revision, the authors used OpenAI ChatGPT, OpenAI image-generation/editing tools, and Codex for manuscript organization, language editing, explanatory-figure drafting/layout, software-code review, test and documentation preparation, and consistency checking. The authors reviewed, edited, and validated all AI-assisted outputs and take full responsibility for the publication's content.

References

[1] Farrar CR, Worden K. An introduction to structural health monitoring. Philos Trans A Math Phys Eng Sci. 2007;365(1851):303-~~15. https://doi.org/10.1098/rsta.2006.1928.~~315. https://doi.org/10.1098/rsta.2006.1928.

[2] Lynch JP, Loh KJ. A summary review of wireless sensors and sensor networks for structural health monitoring. Shock Vib Dig. 2006;38(2):91-128. ~~https://doi.org/10.1177/0583102406061499.~~https://doi.org/10.1177/0583102406061499.

[3] Hassani S, Dackermann U. A systematic review of advanced sensor technologies for non-destructive testing and structural health monitoring. Sensors. 2023;23(4):2204. ~~https://doi.org/10.3390/s23042204.~~https://doi.org/10.3390/s23042204.

[4] Bao Y, Li H. Machine learning paradigm for structural health monitoring. Struct Health Monit. 2021;20(4):1353-~~72. https://doi.org/10.1177/1475921720972416.~~1372. https://doi.org/10.1177/1475921720972416.

[5] Azimi M, Eslamlou AD, Pekcan G. Data-driven structural health monitoring and damage detection through deep learning: state-of-the-art review. Sensors. 2020;20(10):2778. ~~https://doi.org/10.3390/s20102778.~~https://doi.org/10.3390/s20102778.

[6] Gomez-Cabrera A, Escamilla-Ambrosio PJ. Review of machine-learning techniques applied to structural health monitoring systems for building and bridge structures. Appl Sci. 2022;12(21):10754. ~~https://doi.org/10.3390/app122110754.~~https://doi.org/10.3390/app122110754.

[7] ~~Bröring~~Broering A, Echterhoff J, Jirka S, Simonis I, Everding T, Stasch C, et al. New generation Sensor Web Enablement. Sensors. 2011;11(3):2652-~~99. https://doi.org/10.3390/s110302652.~~2699. https://doi.org/10.3390/s110302652.

[8] Liang S, ~~Huang CY,~~ Khalafbeigi T, van der Schaaf H, editors. OGC SensorThings API Part 1: Sensing~~, version~~ Version 1.~~0~~1. OGC Implementation Standard ~~15-078r6~~18-088. Open Geospatial Consortium; ~~2016. https://docs.ogc.org/is/15-078r6/15-078r6.html~~2021. https://docs.ogc.org/is/18-088/18-088.html [accessed ~~25 July~~1 September 2026].

[9] Cugola G, Margara A. Processing flows of information: from data stream to complex event processing. ACM Comput Surv. 2012;44(3):15. ~~https://doi.org/10.1145/2187671.2187677.~~https://doi.org/10.1145/2187671.2187677.

[10] Zhao D, Wang Z, Wang J, Liao J, Li Z, Yang S. GMFAgent: domain knowledge-driven agent for ground motion field estimation. SoftwareX. 2026;34:102673. ~~https://doi.org/10.1016/j.softx.2026.102673.~~https://doi.org/10.1016/j.softx.2026.102673.

[11] Yang S, Li M, Liao J, Wang J, Zhao D, Li Z, et al. Predictive-SHM: an open-source, extensible software toolkit for multi-sensor structural health monitoring and time-series prediction. SoftwareX. 2026;35:102732. ~~https://doi.org/10.1016/j.softx.2026.102732.~~https://doi.org/10.1016/j.softx.2026.102732.

[12] Lim B, Zohren S. Time-series forecasting with deep learning: a survey. Philos Trans A Math Phys Eng Sci. 2021;379(2194):20200209. ~~https://doi.org/10.1098/rsta.2020.0209.~~https://doi.org/10.1098/rsta.2020.0209.

[13] Kong X, Chen Z, Liu W, Ning K, Zhang L, Marier SM, et al. Deep learning for time series forecasting: a survey. Int J Mach Learn Cybern. 2025;16:5079-5112. ~~https://doi.org/10.1007/s13042-025-02560-w.~~https://doi.org/10.1007/s13042-025-02560-w.

[14] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. Adv Neural Inf Process Syst. 2017;30:5998-6008.

[15] Oreshkin BN, Carpov D, Chapados N, Bengio Y. N-BEATS: neural basis expansion analysis for interpretable time series forecasting. In: International Conference on Learning Representations; 2020.

[16] Lim B, ~~Arik~~Arik SO, Loeff N, Pfister T. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. Int J Forecast. 2021;37(4):1748-~~64. https://doi.org/10.1016/j.ijforecast.2021.03.012.~~1764. https://doi.org/10.1016/j.ijforecast.2021.03.012.

[17] Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, et al. Informer: beyond efficient Transformer for long sequence time-series forecasting. Proc AAAI Conf Artif Intell. 2021;35(12):11106-~~15. https://doi.org/10.1609/aaai.v35i12.17325.~~11115. https://doi.org/10.1609/aaai.v35i12.17325.

[18] Wu H, Xu J, Wang J, Long M. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. Adv Neural Inf Process Syst. 2021;34:22419-~~30~~22430.

[19] Nie Y, Nguyen NH, Sinthong P, Kalagnanam J. A time series is worth 64 words: long-term forecasting with Transformers. In: International Conference on Learning Representations; 2023.

[20] Mitchell M, Wu S, Zaldivar A, Barnes P, Vasserman L, Hutchinson B, et al. Model cards for model reporting. In: Proceedings of the Conference on Fairness, Accountability, and Transparency; 2019. p. ~~220-9. https://doi.org/10.1145/3287560.3287596.~~220-229. https://doi.org/10.1145/3287560.3287596.

[21] Gebru T, Morgenstern J, Vecchione B, Vaughan JW, Wallach H, ~~Daumé~~Daume H III, et al. Datasheets for datasets. Commun ACM. 2021;64(12):86-92. ~~https://doi.org/10.1145/3458723.~~https://doi.org/10.1145/3458723.

[22] Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. Sci Data. 2016;3:160018. ~~https://doi.org/10.1038/sdata.2016.18.~~https://doi.org/10.1038/sdata.2016.18.

[23] Barker M, Chue Hong NP, Katz DS, Lamprecht AL, Martinez-Ortiz C, Psomopoulos F, et al. Introducing the FAIR Principles for research software. Sci Data. 2022;9:622. ~~https://doi.org/10.1038/s41597-022-01710-x.~~https://doi.org/10.1038/s41597-022-01710-x.

[24] Smith AM, Katz DS, Niemeyer KE; FORCE11 Software Citation Working Group. Software citation principles. PeerJ Comput Sci. 2016;2:e86. ~~https://doi.org/10.7717/peerj-cs.86.~~https://doi.org/10.7717/peerj-cs.86.

[25] Moreau L, Missier P, editors. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium; 2013. ~~https://www.w3.org/TR/2013/REC-prov-dm-20130430/~~https://www.w3.org/TR/2013/REC-prov-dm-20130430/ [accessed 25 July 2026].

[26] Wilson G, Aruliah DA, Brown CT, Chue Hong NP, Davis M, Guy RT, et al. Best practices for scientific computing. PLoS Biol. 2014;12(1):e1001745. ~~https://doi.org/10.1371/journal.pbio.1001745.~~https://doi.org/10.1371/journal.pbio.1001745.

[27] Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten simple rules for reproducible computational research. PLoS Comput Biol. 2013;9(10):e1003285. ~~https://doi.org/10.1371/journal.pcbi.1003285.~~https://doi.org/10.1371/journal.pcbi.1003285.

[28] Pineau J, Vincent-Lamarre P, Sinha K, Lariviere V, Beygelzimer A, d'Alche-Buc F, et al. Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. J Mach Learn Res. 2021;22(164):1-20.

[29] Lamprecht AL, Garcia L, Kuzak M, Martinez C, Arcila R, Martin Del Pico E, et al. Towards FAIR principles for research software. Data Sci. 2020;3(1):37-59. ~~https://doi.org/10.3233/DS-190026.~~https://doi.org/10.3233/DS-190026.

[30] Stodden V, McNutt M, Bailey DH, Deelman E, Gil Y, Hanson B, et al. Enhancing reproducibility for computational methods. Science. 2016;354(6317):1240-~~1. https://doi.org/10.1126/science.aah6168.~~1241. https://doi.org/10.1126/science.aah6168.